

# LABORATORY OBSERVATION/RECORD

**Course:** Full Stack Web Development

**Course Code:** 23CM4121

**Branch:** CSM

**Section:** B

**Task No.:** 2

**Student Name:** M. Niharika

**Roll No.:** A24126552097

---

## 1. TITLE

**Build an Interactive Student Profile Generator Using JavaScript DOM Manipulation**

---

## 2. AIM

To build an interactive webpage that collects student details through an HTML form and dynamically generates a styled profile card using **JavaScript DOM manipulation**, without reloading the page.

---

## 3. OBJECTIVE

The objectives of this experiment are:

- To understand how to structure a form using HTML5 input elements.
  - To use a JavaScript class to model a real-world entity such as a Student.
  - To select DOM elements using `document.getElementById()`.
  - To handle form submission events using `addEventListener()` and `preventDefault()`.
  - To read values typed into form fields using the `.value` property.
  - To dynamically create and style elements using `document.createElement()`.
  - To append dynamically created elements to the page using `appendChild()`.
  - To iterate over an array of fields using `forEach()` to avoid repetitive code.
  - To apply CSS3 styling for a clean, card-based interface with hover and focus effects.
- 

## 4. THEORY

A **JavaScript class** is a blueprint for creating objects that share the same shape. The constructor runs once when a new object is created with the `new` keyword, and sets up its properties — here, a Student's name, roll number, department, and CGPA.

The **DOM (Document Object Model)** represents the page as a tree of elements that JavaScript can read and change. Instead of hardcoding the profile card in the HTML, this program builds it piece by piece at runtime, based on whatever the user typed into the form.

The key methods and concepts used in this program are:

- `addEventListener("submit", fn)` runs a function every time the form is submitted, instead of the browser's default reload-and-navigate behaviour.
- `e.preventDefault()` stops that default page reload, so the form can be handled entirely with JavaScript.
- `document.getElementById()` selects a specific element from the page by its id attribute.
- `document.createElement()` and `appendChild()` create a new, empty element in memory and then attach it into the visible page.
- `fields.forEach(...)` loops through an array of label/value pairs to build the repeated paragraph rows without repeating code.
- `:focus` styles an input only while it is actively selected, giving the user visual feedback while typing.

The basic flow of the page is:

User fills the form → submit event fires → `preventDefault()` stops the reload → input values are read → a Student object is created → a profile card is built with DOM methods → the card is appended to the page

---

## 5. ALGORITHM / PROCEDURE

1. Create two files: index.html and script.js, and link script.js at the end of the body.
2. Add a form with labelled inputs for Name, Roll No, Department, and CGPA, plus a submit button.
3. Add an empty container `<div>` where the generated profile card will be inserted.
4. In script.js, define a Student class with a constructor that stores name, rollNo, department, and cgpa.
5. Select the form and the container element using `document.getElementById()`.
6. Add a submit event listener on the form.
7. Inside the handler, call `e.preventDefault()` to stop the default page reload.
8. Read the current values from each input field using `.value`.
9. Create a new Student object using the values just read.
10. Clear any existing content in the container using `container.innerHTML = ""`.
11. Create a card `<div>` and a heading element for the profile card.
12. Build an array of label/value pairs from the student object's properties.
13. Loop through the array with `forEach()` to create a paragraph and label span for each field.
14. Append the card to the container so it appears on the page.
15. Save the files and open index.html in a browser to test with different input values.

---

## 6. PROGRAM

**index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Student Profile Generator</title>
<style>
```

```

body { font-family: Arial, sans-serif; text-align: center; padding: 40px; background: #f2f4f8;
}

.form-card { margin: 0 auto; background: #fff; border-radius: 10px;
  box-shadow: 0 4px 14px rgba(0,0,0,0.12); padding: 25px 35px;
  width: 300px; text-align: left; }
.form-card h2 { color: #2f6fed; border-bottom: 2px solid #2f6fed; padding-bottom: 8px;
margin-top: 0; }
.form-card label { display: block; font-weight: 600; margin: 14px 0 6px; }
.form-card input {
  width: 100%; padding: 8px 10px; border: 1px solid #ccc; border-radius: 6px;
  font-size: 14px; box-sizing: border-box;
}
.form-card input:focus { outline: none; border-color: #2f6fed; }

#showProfileBtn { background: #2f6fed; color: #fff; border: none; padding: 12px 28px;
  font-size: 16px; border-radius: 6px; cursor: pointer; margin-top: 20px;
width: 100%; }
#showProfileBtn:hover { background: #1e54c4; }

.profile-card { margin: 30px auto 0; background: #fff; border-radius: 10px;
  box-shadow: 0 4px 14px rgba(0,0,0,0.12); padding: 25px 35px;
  width: 300px; text-align: left; }
.profile-card h2 { color: #2f6fed; border-bottom: 2px solid #2f6fed; padding-bottom: 8px; }
.profile-card span.label { font-weight: 600; display: inline-block; width: 110px; }
</style>
</head>
<body>

<form id="studentForm" class="form-card">
  <h2>Enter Student Details</h2>

  <label for="nameInput">Name</label>
  <input type="text" id="nameInput" required placeholder="e.g. Niharika">

  <label for="rollInput">Roll No</label>
  <input type="text" id="rollInput" required placeholder="e.g. A24126552097">

  <label for="deptInput">Department</label>
  <input type="text" id="deptInput" required placeholder="e.g. CSE (AI & ML)">

  <label for="cgpaInput">CGPA</label>
  <input type="number" id="cgpaInput" step="0.01" min="0" max="10" required placeholder="e.g.
8.88">

  <button type="submit" id="showProfileBtn">Show Student Profile</button>
</form>

<div id="profileContainer"></div>

<script src="script.js"></script>
</body>
</html>

```

## script.js

```
// Student class
class Student {
  constructor(name, rollNo, department, cgpa) {
    this.name = name;
    this.rollNo = rollNo;
    this.department = department;
    this.cgpa = cgpa;
  }
}

// DOM selection
const form = document.getElementById("studentForm");
const container = document.getElementById("profileContainer");

// Event handling - build the object from form input on submit
form.addEventListener("submit", function (e) {
  e.preventDefault(); // stop the page from reloading on submit

  // Read the values the user typed in
  const name = document.getElementById("nameInput").value;
  const rollNo = document.getElementById("rollInput").value;
  const department = document.getElementById("deptInput").value;
  const cgpa = document.getElementById("cgpaInput").value;

  // Create the object with the user-provided values
  const student1 = new Student(name, rollNo, department, cgpa);

  container.innerHTML = "";

  const card = document.createElement("div");
  card.className = "profile-card";

  const title = document.createElement("h2");
  title.textContent = "Student Profile";
  card.appendChild(title);

  const fields = [
    { label: "Name", value: student1.name },
    { label: "Roll No", value: student1.rollNo },
    { label: "Department", value: student1.department },
    { label: "CGPA", value: student1.cgpa }
  ];

  fields.forEach(field => {
    const p = document.createElement("p");
    const label = document.createElement("span");
    label.className = "label";
    label.textContent = field.label + " :";
    p.appendChild(label);
    p.appendChild(document.createTextNode(field.value));
    card.appendChild(p);
  });
});
```

```
container.appendChild(card);  
});
```

---

## 7. CODE EXPLANATION

| Code                                  | Explanation                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| class Student {<br>constructor(...) { | Defines a blueprint for creating student objects with name, rollNo, department, and cgpa. |
| new Student(...)                      | Creates a new object instance using the values read from the form.                        |
| document.getElementById()             | Selects a specific element from the page by its id attribute.                             |
| addEventListener("submit", fn)        | Runs the given function whenever the form is submitted.                                   |
| e.preventDefault()                    | Stops the form's default behaviour of reloading the page on submit.                       |
| .value                                | Reads the current text typed into an input field.                                         |
| container.innerHTML = ""              | Clears any previously generated profile card before adding a new one.                     |
| document.createElement()              | Creates a new, empty HTML element in memory.                                              |
| element.textContent                   | Sets the visible text inside an element.                                                  |
| appendChild()                         | Attaches a created element as a child of another element in the DOM.                      |
| fields.forEach(...)                   | Loops through an array of field objects to build the repeated paragraph markup.           |
| :focus                                | Styles an input only while it is actively selected or being typed into.                   |

---

## 8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and the form was tested with different inputs; the results were recorded below.

| Test Case | Action | Expected Result | Actual Result | Status |
|-----------|--------|-----------------|---------------|--------|
|-----------|--------|-----------------|---------------|--------|

|       |                                                     |                                                                                   |                                  |      |
|-------|-----------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------|------|
| TC-01 | Open index.html in a browser                        | Empty form displayed with 4 labelled fields and a submit button                   | Form displayed correctly         | Pass |
| TC-02 | Leave the Name field empty and click Submit         | Browser blocks submission and prompts for the field                               | Required-field prompt shown      | Pass |
| TC-03 | Fill all fields and click 'Show Student Profile'    | Profile card appears below the form with the entered values; page does not reload | Profile card displayed correctly | Pass |
| TC-04 | Enter a CGPA above 10                               | Browser blocks submission (max=10 validation)                                     | Submission blocked as expected   | Pass |
| TC-05 | Submit the form a second time with different values | Old profile card is removed and replaced with the new one                         | Card replaced correctly          | Pass |
| TC-06 | Click into any input field                          | Border colour changes to blue on focus                                            | Focus effect displayed           | Pass |

### Additional Validation Test

| Test Case | Input/Action                                        | Expected Result                                                        | Status                                                                         |
|-----------|-----------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| TC-07     | Hover over the submit button                        | Background changes to a darker blue                                    | Pass                                                                           |
| TC-08     | Enter a long department name (e.g. 'CSE (AI & ML)') | The 'Department :' label should stay on one line like the other labels | Fail – label wraps onto two lines because the fixed label width (110px) is too |

|  |  |  |                  |
|--|--|--|------------------|
|  |  |  | narrow<br>for it |
|--|--|--|------------------|

# 9. OUTPUT

Output 1: Empty Student Details Form

### Enter Student Details

**Name**

e.g. Niharika

**Roll No**

e.g. A24126552097

**Department**

e.g. CSE (AI & ML)

**CGPA**

e.g. 8.88

Show Student Profile

Output 2: Filled Form and Generated Profile Card (after Submit)

### Enter Student Details

**Name**

**Roll No**

**Department**

**CGPA**

Show Student Profile

### Student Profile

**Name :** Niharika

**Roll No :** A24126552097

**Department :** CSE (AI & ML)

**CGPA :** 8.88

---

## 10. RESULT

Thus, an interactive student profile generator was successfully built using an HTML form and JavaScript DOM manipulation. Submitting the form with valid details correctly created a Student object and rendered a styled profile card without reloading the page. One minor cosmetic issue was observed: the 'Department :' label wraps onto two lines because the label column has a fixed width of 110px, which is too narrow for longer field names; widening it to around 130px would resolve this.